

SPRINT 05: MONGODB

SANTIAGO ÁLVAREZ SALVADO

Contenido

Nivel 1.....	3
Ejercicio 1	10
Muestra los 2 primeros comentarios que aparecen en la base de datos.	10
¿Cuántos usuarios tenemos registrados?	11
¿Cuántos cines existen en el estado de California?	12
¿Cuál fue el primer usuario en registrarse?	13
¿Cuántas películas de comedia existen en nuestra base de datos?	14
Ejercicio 2	15
Ejercicio 3	16
Nivel 2.....	17
Ejercicio 1	17
Ejercicio 2	18
Nivel 3.....	19
Ejercicio 1	19
Ejercicio 2	20

Trabajaremos con una base de datos que contiene colecciones relacionadas con una aplicación de entretenimiento cinematográfico:

users: Almacena información de usuarios/as, incluyendo nombres, emails y contraseñas cifradas.

theatres: Contiene datos de cines, como ID, ubicación (dirección y coordenadas geográficas).

sesiones: Guarda sesiones de usuario, incluyendo ID de usuario y tokens JWT para la autenticación.

movies: Incluye detalles de películas, como trama, géneros, duración, elenco, comentarios, año de lanzamiento, directores, clasificación y premios.

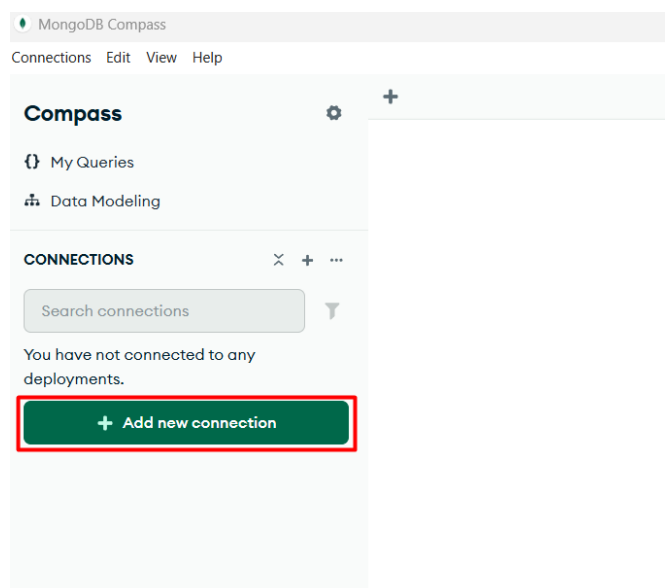
comments: Almacena comentarios de usuarios/as sobre películas, con información del autor/a del comentario, ID de la película, texto del comentario y la fecha.

Realizarás algunas consultas que te pide el cliente/a, quien está midiendo si serás capaz o no de hacerte cargo de la parte analítica del proyecto vinculado con su base de datos.

Nivel 1

Crea una base de datos con MongoDB utilizando como colecciones los archivos adjuntos.

Lo primero que hay que hacer es crear una conexión a un servidor, en este caso se hará localmente, para ello será necesario darle a add new connection en la pantalla principal de mongodb



Se abrirá una ventana en la que tendremos que poner la URI, como no hay servidor habrá que realizar una conexión local con localhost, en name se pondrá el nombre que corresponda, y se guarda en save & connect

New Connection

Manage your connection settings

URI ⓘ

Edit Connection String

mongodb://localhost:27017/

Name

ServidorItAcademy

Color

No Color

☐ Favorite this connection
Favoriting a connection will pin it to the top of your list of connections

[Advanced Connection Options](#)

How do I find my connection string in Atlas?
If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.
[See example](#)

How do I format my connection string?
[See example](#)

Cancel

Save

Connect

Save & Connect

Una vez realizada la conexión aparecerá lo siguiente

Compass

My Queries

Data Modeling

CONNECTIONS (1)

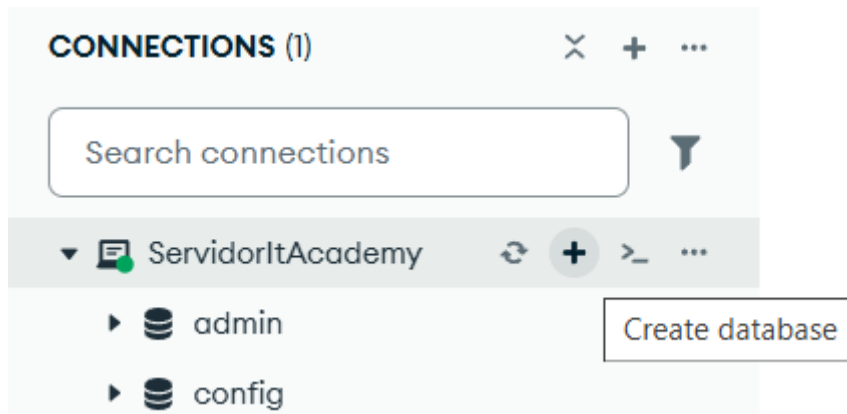
ServidorItAcademy

admin

config

local

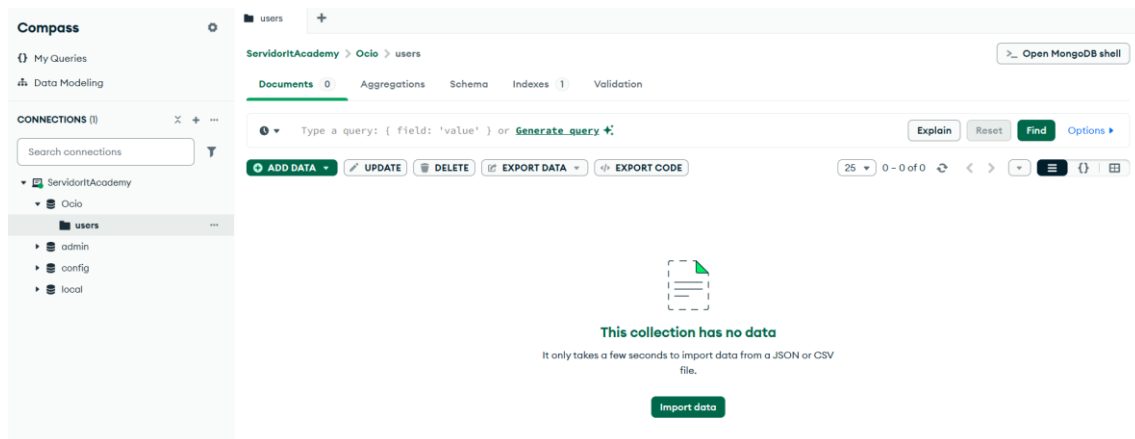
Para crear la base de datos hay que darle al + que aparece a la derecha de ServidorItAcademy, esto abrirá una ventana en la que será necesario configurar la base de datos



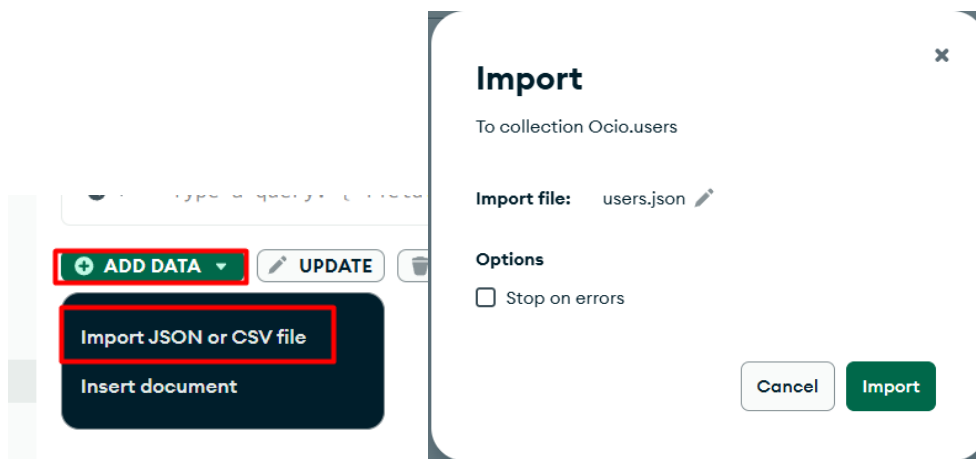
Se abrirá una ventana en la que será necesario definir el nombre y al menos una colección, como hay 5 colecciones con las que se va a trabajar se pondrá users.json por ejemplo, las 3 opciones de abajo no serán necesarias, una vez definidos los nombres se le dará a create database.

A screenshot of the 'Create Database' dialog box in MongoDB Compass. The dialog has a title bar with a close button. It contains fields for 'Database Name' (Ocio) and 'Collection Name' (users). Below these are three checkboxes: 'Time-Series', 'Use Custom Collation', and 'Clustered Collection', each with a description and a 'Learn More' link. At the bottom are 'Cancel' and 'Create Database' buttons.

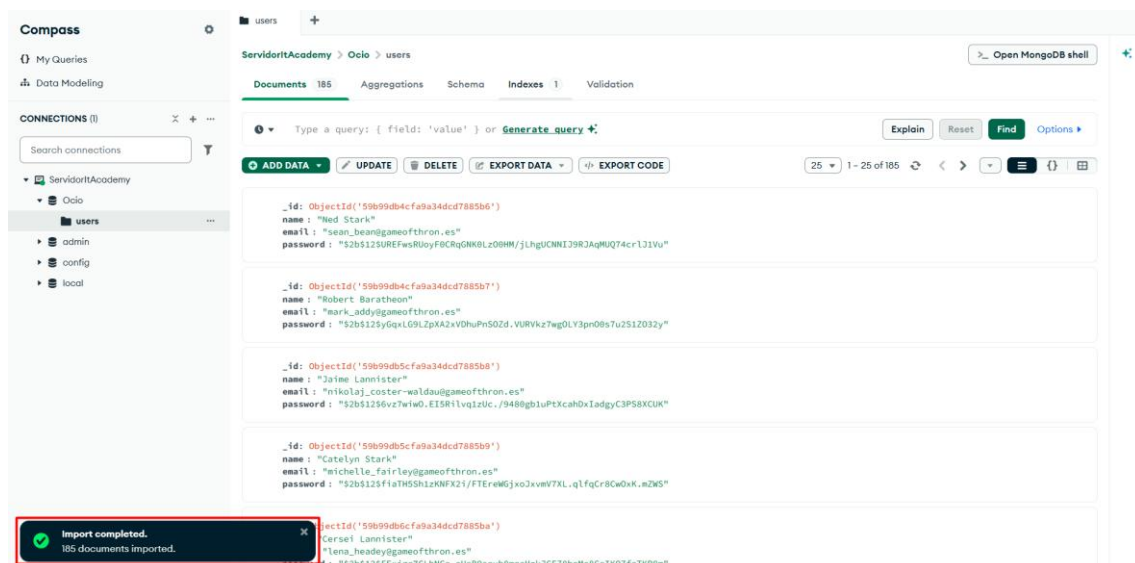
Cuando se haya creado la base de datos aparecerá la siguiente ventana



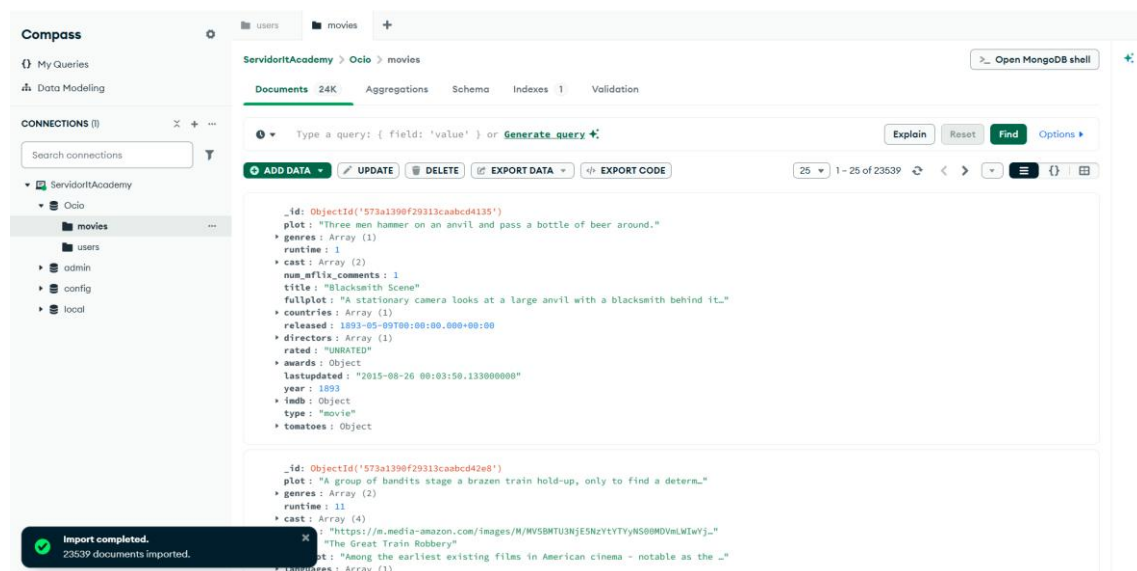
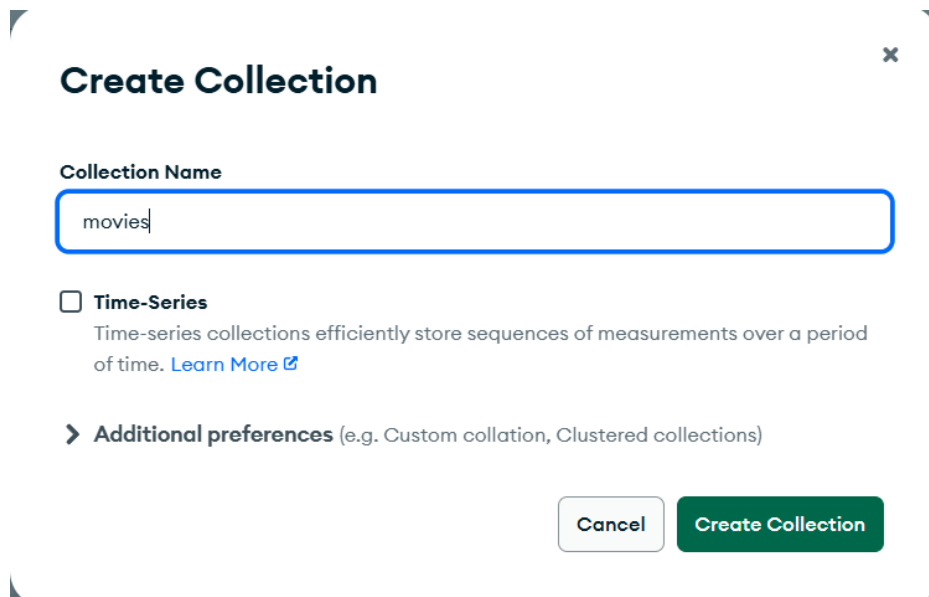
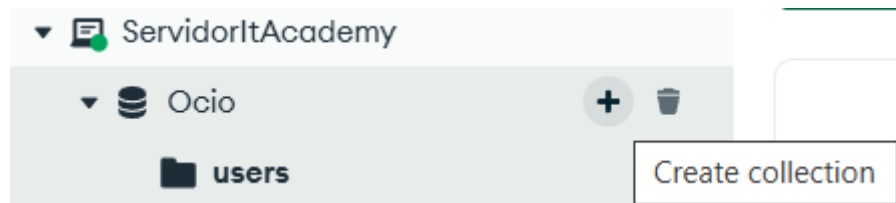
Para añadir el archivo users.json habrá que darle a add data → import json or csv file y seleccionar el archivo users.json e import.



Al importarlo aparecerán todos los datos del json y un mensaje en la parte inferior izquierda en la que pondrá que la importación se ha completado y la cantidad de registros importados



Para importar el resto de archivos será necesario darle al + en la base de dato Ocio para crear una colección nueva, ponerle un nombre adecuado y cargar el json correspondiente dándole a add data → import json or csv file.



Create Collection

Collection Name

theaters

☐ Time-Series

Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

➤ Additional preferences (e.g. Custom collation, Clustered collections)

Cancel

Create Collection

The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel lists the 'ServidoritAcademy' database with collections 'Ocio', 'movies', 'theaters', 'users', 'admin', 'config', and 'local'. The 'theaters' collection is selected. The main panel shows the 'Documents' tab for the 'theaters' collection, displaying 16K documents. A sample document is shown:

```
{ "_id": ObjectId("59a47286cf9a3a73e51e72c"), "theaterId": 1000, "location": Object }
```

. At the bottom, a green notification bar states 'Import completed. 1564 documents imported.'

Create Collection

Collection Name

sessions

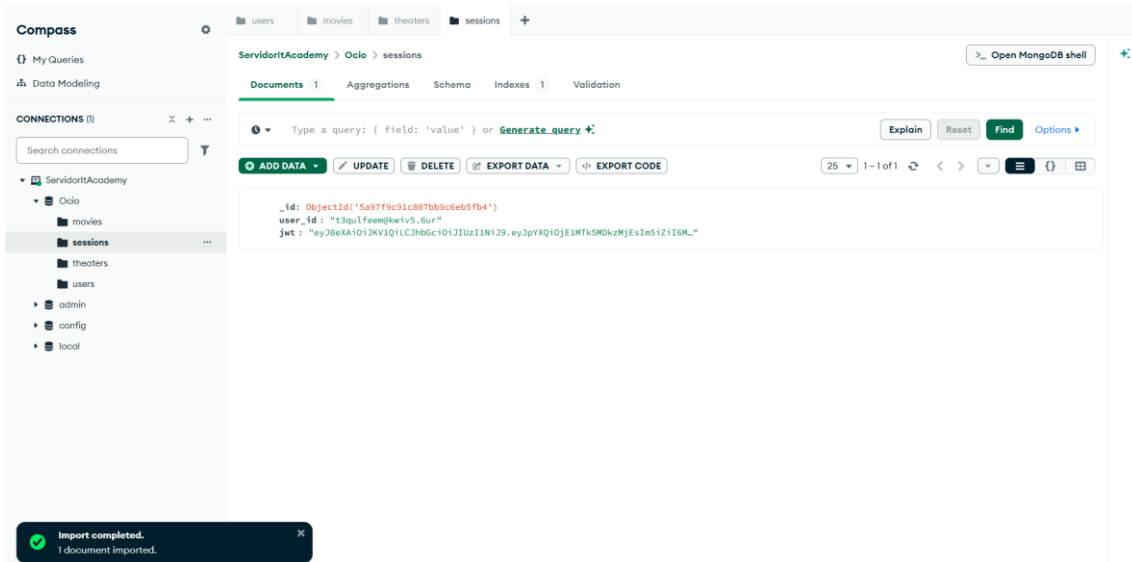
☐ Time-Series

Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

➤ Additional preferences (e.g. Custom collation, Clustered collections)

Cancel

Create Collection



Create Collection

Collection Name

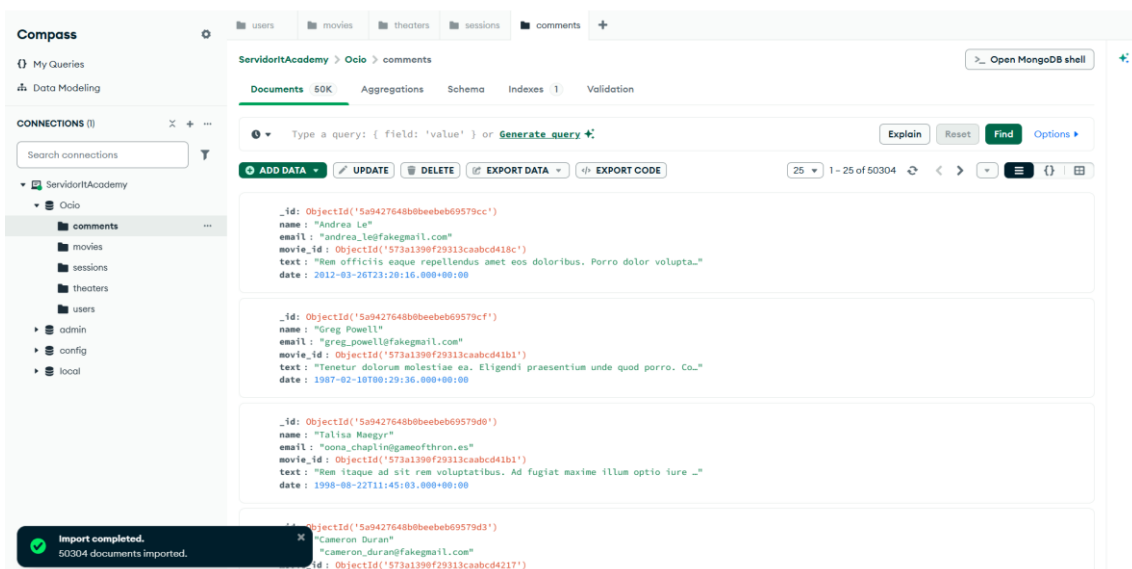
☐ Time-Series

Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

➤ Additional preferences (e.g. Custom collation, Clustered collections)

Cancel

Create Collection



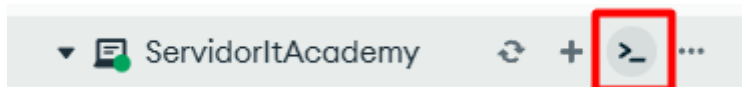
Ejercicio 1

Muestra los 2 primeros comentarios que aparecen en la base de datos.

Para poder mostrarlo hay que poner en la linea de arriba que pone type query los simbolos {} y en el campo limit 2, para que solo aparezcan los dos primeros comentarios

The screenshot shows the MongoDB Compass interface. At the top, there's a query editor with a JSON query: `{}`. Below it, the 'Project' section shows `{ field: 0 }`, the 'Sort' section shows `{ field: -1 } or [['field', -1]]`, the 'Collation' section shows `{ locale: 'simple' }`, and the 'Index Hint' section shows `{ field: -1 } or 'indexName'`. On the right, there are input fields for 'Skip' (0) and 'Limit' (2), and a 'Max Time MS' field (60000). Below the query editor, there are buttons for 'ADD DATA', 'UPDATE', 'DELETE', 'EXPORT DATA', and 'EXPORT CODE'. At the bottom, there are two document preview cards. The first card shows a document with fields: `_id`, `name`, `email`, `movie_id`, `text`, and `date`. The second card shows a similar document with different values.

También se podría hacer con la terminal, para abrir la terminal hay que darle al símbolo de la conexión al servidor

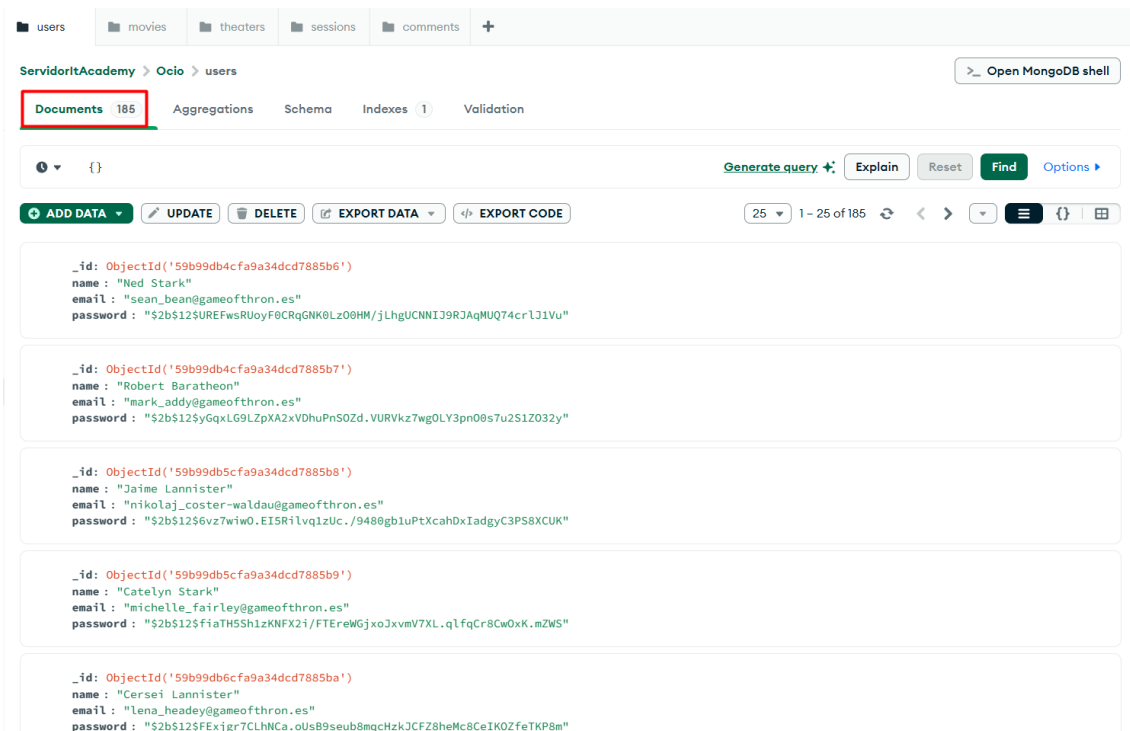


Una vez dentro habrá que poner `use Ocio` para poder trabajar con la base de datos que hemos creado, para poder ver los dos primeros comentarios que aparecen en la base de datos habrá que poner `db.comments.find().limit(2)`

```
>_MONGOSH
> use Ocio
< switched to db Ocio
> db.comments.find().limit(2)
< {
  _id: ObjectId('5a9427648b0beeb69579cc'),
  name: 'Andrea Le',
  email: 'andrea_le@fakegmail.com',
  movie_id: ObjectId('573a1390f29313caabcd418c'),
  text: 'Rem officiiis eaque repellendus amet eos doloribus. Porro dolor voluptatum voluptates neque culpa molestias. Voluptate unde nulla temporibu
  date: 2012-03-26T23:20:16.000Z
}
{
  _id: ObjectId('5a9427648b0beeb69579cf'),
  name: 'Greg Powell',
  email: 'greg_powell@fakegmail.com',
  movie_id: ObjectId('573a1390f29313caabcd41b1'),
  text: 'Tenetur dolorum molestiae ea. Eligendi praesentium unde quod porro. Commodi nisi sit placeat rerum vero cupiditate neque. Dolorum nihil ve
  date: 1987-02-10T00:29:36.000Z
}
Ocio>
```

¿Cuántos usuarios tenemos registrados?

Para ver cuantos usuarios hay registrados hay que abrir users y en la parte de arriba donde pone document aparecerá un numero, ese numero es el total de registros, por lo que en este caso hay 185 usuarios registrados.



The screenshot shows the MongoDB Atlas web interface. At the top, there's a navigation bar with tabs for 'users', 'movies', 'theaters', 'sessions', 'comments', and a plus icon. Below this, the breadcrumb 'ServidorItAcademy > Ocio > users' is visible. A button 'Open MongoDB shell' is on the right. The 'Documents' tab is selected, and the count '185' is highlighted with a red box. Below the count, there are tabs for 'Aggregations', 'Schema', 'Indexes' (with a '1' indicator), and 'Validation'. A query editor is present with a 'Generate query' button and 'Explain', 'Reset', 'Find', and 'Options' buttons. Below the query editor, there are buttons for 'ADD DATA', 'UPDATE', 'DELETE', 'EXPORT DATA', and 'EXPORT CODE'. A pagination bar shows '25' items per page and '1 - 25 of 185' documents. Below the pagination, several document entries are displayed, each with fields like '_id', 'name', 'email', and 'password'.

Para verlo en la terminal habrá que hacer lo mismo que en el caso anterior, hay que darle al símbolo de la terminal de comando en la conexión al servidor, poner use Ocio y el código

```
>_MONGOSH

> use Ocio
< switched to db Ocio

> db.users.countDocuments()
< 185

Ocio>
```

¿Cuántos cines existen en el estado de California?

Para poder dar respuesta a esta pregunta será necesario abrir theaters, en la barra de filter habrá que poner el código `{ "location.address.state": "CA" }`, esto hará que se filtren los cines donde state coincida con CA que es California.

The screenshot shows the MongoDB Compass interface for the 'ServidorItAcademy' database, specifically the 'Ocio' collection 'theaters'. The filter bar at the top contains the query `{ "location.address.state": "CA" }`, which is highlighted with a red box. Below the filter bar, the results are displayed in a list format, showing the first five documents. Each document is a JSON object representing a theater, with fields like `_id`, `theaterId`, `location`, `address`, and `geo`. The `address` field is expanded for the first document, showing `street1`, `city`, `state`, and `zipcode`. The `state` field is highlighted in green, indicating it matches the filter. The pagination bar at the bottom shows '1 - 25 of 169' results, also highlighted with a red box.

Para verlo en la terminal se hará igual que en los casos anteriores, poniendo el código `db.theaters.find({ "location.address.state": "CA" })`

```
>_MONGOSH
> use Ocio
< switched to db Ocio
> db.theaters.find({ "location.address.state": "CA" })
< {
  _id: ObjectId('59a47286cfa9a3a73e51e72e'),
  theaterId: 1008,
  location: {
    address: {
      street1: '1621 E Monte Vista Ave',
      city: 'Vacaville',
      state: 'CA',
      zipcode: '95688'
    },
    geo: {
      type: 'Point',
      coordinates: [
        -121.96328,
        38.367649
      ]
    }
  }
}
{
  _id: ObjectId('59a47286cfa9a3a73e51e734'),
  theaterId: 1009,
  location: {
    address: {
      street1: '6310 E Pacific Coast Hwy',
      city: 'Long Beach',
```

¿Cuál fue el primer usuario en registrarse?

Para saber cuál es el primer usuario en registrarse en los filtros habrá que poner sort { _id: 1 } y limit 1, de esta manera aparecerá un solo usuario que será el que se registro primero

The screenshot shows a MongoDB query builder interface. At the top, there's a text input for a query: "Type a query: { field: 'value' } or [Generate query](#)". To the right are buttons for "Explain", "Reset", "Find", and "Options". Below this, the query is built using a visual interface. The "Sort" field is set to "{ _id: 1 }" and is highlighted with a red box. The "Limit" field is set to "1" and is also highlighted with a red box. Other fields include "Project" ("{ field: 0 } "), "Collation" ("{ locale: 'simple' } "), "Index Hint" ("{ field: -1 } or 'indexName' "), "Max Time MS" ("60000"), and "Skip" ("0"). At the bottom, there are buttons for "ADD DATA", "UPDATE", "DELETE", "EXPORT DATA", and "EXPORT CODE". Below the buttons, the query result is displayed as a JSON object:

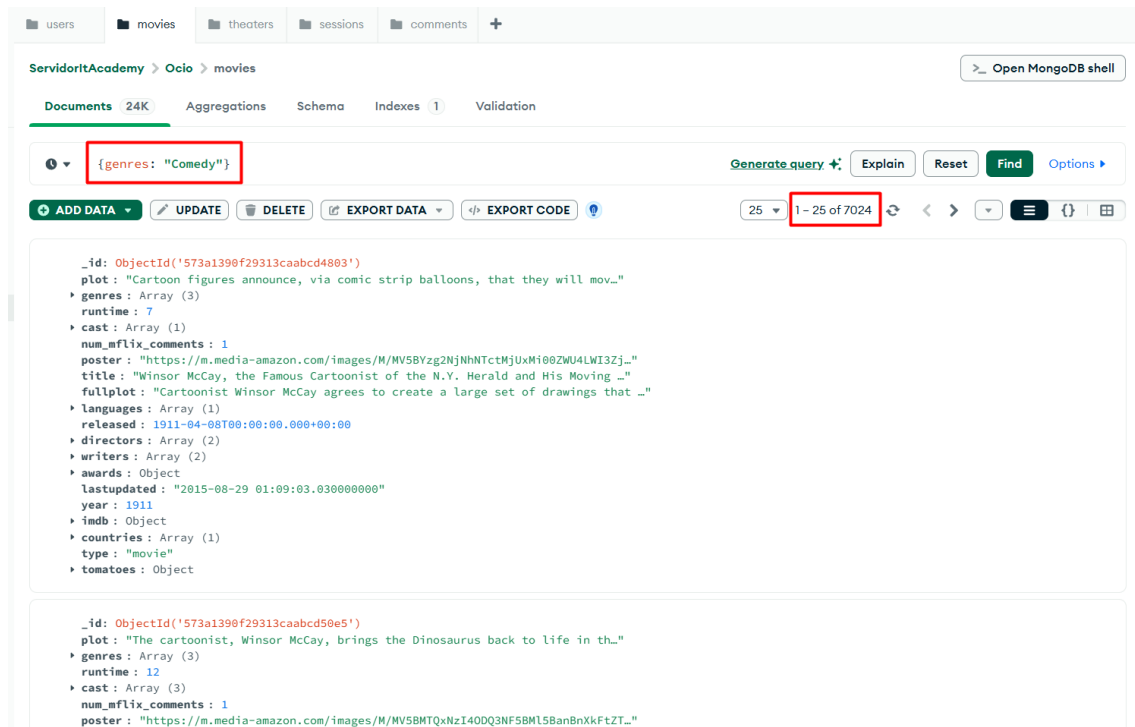
```
{
  "_id": ObjectId('59b99db4cfa9a34dcd7885b6'),
  "name": "Ned Stark",
  "email": "sean_bean@gameofthron.es",
  "password": "$2b$12$UREFwsRUoyF0CRqGNK0Lz00HM/jLhgUCNNI39RJAqMUQ74cr1J1Vu"
}
```

Para verlo en la consola de comandos tendremos que poner el código `db.users.find({ _id: 1 }).sort().limit(1)`

```
> _MONGOSH
> use Ocio
< switched to db Ocio
> db.users.find().sort({ _id: 1 }).limit(1)
< {
  _id: ObjectId('59b99db4cfa9a34dcd7885b6'),
  name: 'Ned Stark',
  email: 'sean_bean@gameofthron.es',
  password: '$2b$12$UREFwsRUoyF0CRqGNK0Lz00HM/jLhgUCNNI39RJAqMUQ74cr1J1Vu'
}
```

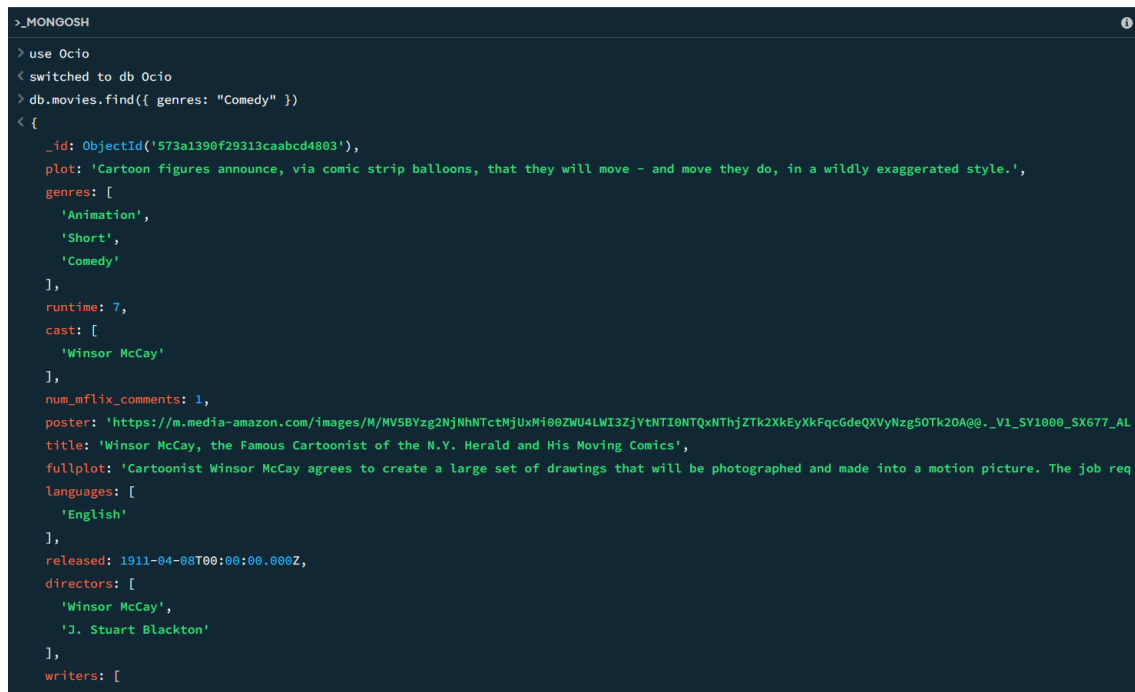
¿Cuántas películas de comedia existen en nuestra base de datos?

Para saber cuántas películas existen en la base de datos habrá que poner en la caja filter {genres: "Comedy"} para que se filtren todas las películas con el género de comedia.



The screenshot shows the MongoDB Compass interface. At the top, there's a navigation bar with tabs for 'users', 'movies', 'theaters', 'sessions', and 'comments'. Below this, the 'movies' collection is selected. The 'Documents' tab is active, showing a list of documents. A query filter '{genres: \"Comedy\"}' is entered in the filter box, which is highlighted with a red rectangle. Below the filter box, there are buttons for 'Generate query', 'Explain', 'Reset', 'Find', and 'Options'. The 'Find' button is highlighted. Below the buttons, there's a pagination bar showing '25' documents per page and '1 - 25 of 7024' documents, with the range '1 - 25 of 7024' highlighted by a red rectangle. The main area displays two document snippets. The first document has fields like '_id', 'plot', 'genres', 'runtime', 'cast', 'num_mflix_comments', 'poster', 'title', 'fullplot', 'languages', 'released', 'directors', 'writers', 'awards', 'lastupdated', 'year', 'imdb', 'countries', 'type', and 'tomatoes'. The second document is partially visible below it.

Para hacerlo en la consola de comandos habrá que poner `db.movies.find({ genres: "Comedy" })`



```
>_MONGOSH
> use Ocio
< switched to db Ocio
> db.movies.find({ genres: "Comedy" })
< {
  _id: ObjectId('573a1390f29313caabed4803'),
  plot: 'Cartoon figures announce, via comic strip balloons, that they will move ~ and move they do, in a wildly exaggerated style.',
  genres: [
    'Animation',
    'Short',
    'Comedy'
  ],
  runtime: 7,
  cast: [
    'Winsor McCay'
  ],
  num_mflix_comments: 1,
  poster: 'https://m.media-amazon.com/images/M/MV5BYzg2NjNhNTctMjUxMi00ZWU4LWI3ZjYtNTI0NTQxNThjZTk2XkEyXkFqcGdeQXVyNzg5OTk2OAA@._V1_S1000_SX677_AL',
  title: 'Winsor McCay, the Famous Cartoonist of the N.Y. Herald and His Moving Comics',
  fullplot: 'Cartoonist Winsor McCay agrees to create a large set of drawings that will be photographed and made into a motion picture. The job req',
  languages: [
    'English'
  ],
  released: 1911-04-08T00:00:00.000Z,
  directors: [
    'Winsor McCay',
    'J. Stuart Blackton'
  ],
  writers: [
    'Winsor McCay (comic strip "Little Nemo in Slumberland")'
```

Ejercicio 2

Muéstrame todos los documentos de las películas producidas en 1932, pero que el género sea drama o estén en francés.

Para mostrar todos los documentos de las películas con esas condiciones será necesario filtrar por año y poner la condición or para que se seleccione los que tienen género Drama o que el lenguaje sea francés, la sentencia sería la siguiente: `{year: 1932, $or: [{genres: "Drama"}, {languages: "French"}]}`

The screenshot shows the MongoDB Compass web interface. At the top, there's a breadcrumb trail: `ServidorItAcademy > Ocio > movies`. Below this, there are tabs for `Documents` (24K), `Aggregations`, `Schema`, `Indexes` (1), and `Validation`. The `Documents` tab is active. In the search bar, the query `{year: 1932, $or: [{genres: "Drama"}, {languages: "French"}]}` is entered. To the right of the search bar are buttons for `Generate query`, `Explain`, `Reset`, `Find`, and `Options`. Below the search bar, there are buttons for `ADD DATA`, `UPDATE`, `DELETE`, `EXPORT DATA`, and `EXPORT CODE`. The results section shows two documents. The first document is for the movie 'The Blood of a Poet' (1932), and the second is for 'The Blue Light' (1932). Both documents include fields like `_id`, `plot`, `runtime`, `rated`, `cast`, `num_mflix_comments`, `poster`, `title`, `lastupdated`, `languages`, `released`, `directors`, `writers`, `awards`, `year`, `imdb`, `countries`, `type`, and `tomatoes`.

En la consola de comando sería igual pero poniendo delante `db.movies.find` para poder buscar en las películas, la sentencia sería la siguiente: `db.movies.find({year: 1932, $or: [{genres: "Drama" }, {languages: "French" }]} )`

The screenshot shows a MongoDB command prompt. The user has entered the following commands: `> use Ocio`, `< switched to db Ocio`, and `> db.movies.find({year: 1932, $or: [{genres: "Drama" }, {languages: "French" }]} )`. The prompt shows the result of the query, which is a single document for the movie 'The Blood of a Poet' (1932). The document includes fields like `_id`, `plot`, `runtime`, `rated`, `cast`, `num_mflix_comments`, `poster`, `title`, `lastupdated`, `languages`, `released`, `directors`, `writers`, `awards`, `year`, `imdb`, `countries`, `type`, and `tomatoes`.

Ejercicio 3

Muéstrame todos los documentos de películas estadounidenses que tengan entre 5 y 9 premios que fueron producidas entre 2012 y 2014.

Para mostrar todos los documentos de las películas estadounidenses con esas características será necesario filtrar los countries con el valor USA, y hacer como un between en sql para poder poner que los premios ganados sean entre 5 y 9, con los años es igual, \$gte significa mayor o igual a y \$lte significa menor o igual a, la sentencia seria la siguiente: {countries: "USA", "awards.wins": {\$gte: 5, \$lte: 9}, year: {\$gte: 2012, \$lte: 2014}}

The screenshot shows the MongoDB Compass interface. At the top, the database 'ServidoritAcademy' and collection 'Ocio' are selected. The 'Documents' tab is active, showing 24K documents. A query filter is applied: {countries: "USA", "awards.wins": {"\$gte": 5, "\$lte": 9}, year: {"\$gte": 2012, "\$lte": 2014}}. The results show a single document for the movie 'The Secret Life of Walter Mitty'. The document structure includes fields like _id, fullplot, imdb (with year 2013), plot, genres, rated, metacritic, title, lastupdated, languages, writers, type, tomatoes, poster, num_mflix_comments, released, awards (with 5 wins), countries (with USA), cast, directors, and runtime.

En la consola de comando seria igual pero con como abrir map view mongoddb delante de la sentencia para poder buscar en movies, la sentencia seria la siguiente: db.movies.find({countries: "USA", "awards.wins": {\$gte: 5, \$lte: 9}, year: {\$gte: 2012, \$lte: 2014}})

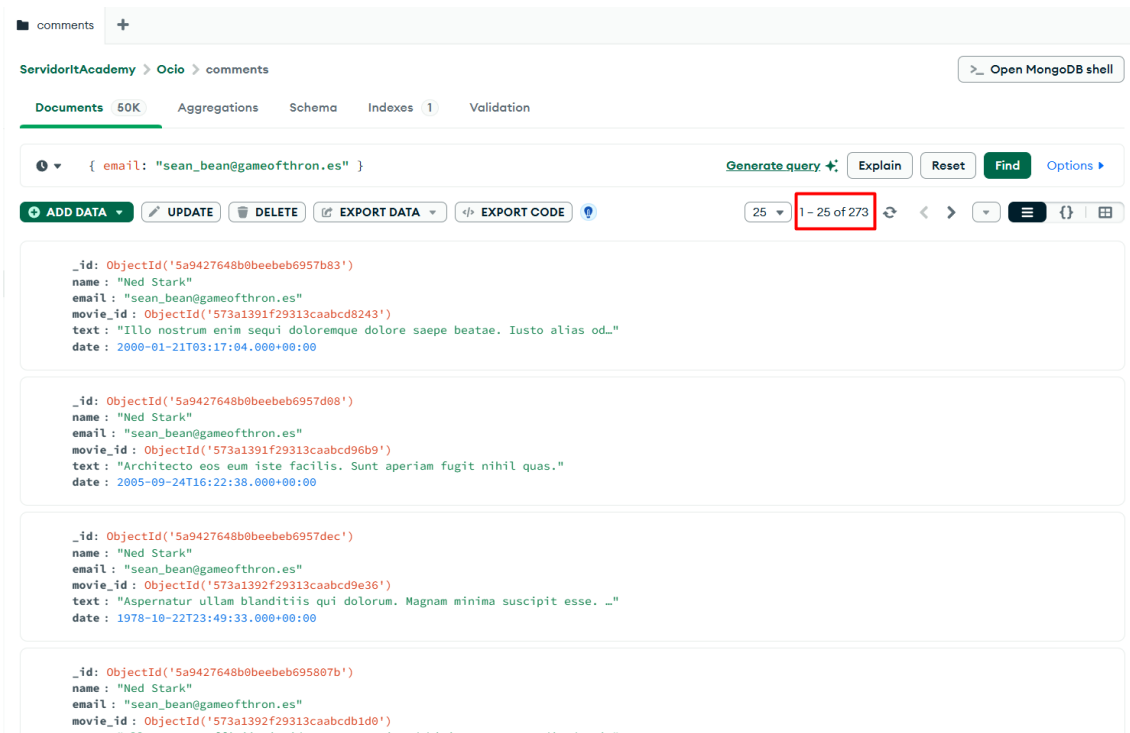
```
>_MONGOSH
> use Ocio
< switched to db Ocio
> db.movies.find({countries: "USA", "awards.wins": {$gte: 5, $lte: 9}, year: {$gte: 2012, $lte: 2014}})
< {
  _id: ObjectId('573a13acf29313caabd29366'),
  fullplot: 'The manager of the negative assets sector of Life magazine, Walter Mitty, has been working for sixteen years for the magazine and has',
  imdb: {
    rating: 7.4,
    votes: 211230,
    id: 359950
  },
  year: 2013,
  plot: 'When his job along with that of his co-worker are threatened, Walter takes action in the real world embarking on a global journey that tur',
  genres: [
    'Adventure',
    'Comedy',
    'Drama'
  ],
  rated: 'PG',
  metacritic: 54,
  title: 'The Secret Life of Walter Mitty',
  lastupdated: '2015-08-31 00:10:51.747000000',
  languages: [
    'English',
    'Spanish',
    'Icelandic'
  ],
  writers: [
    'Steve Conrad (screenplay)'
  ]
}
```


Nivel 2

Ejercicio 1

Cuenta cuántos comentarios escribe un usuario que utiliza "GAMEOFTHRON.ES" como dominio de correo electrónico.

Para saber cuántos comentarios escribe un usuario en concreto, será necesario buscar un usuario con un email que termine con gameofthron.es, y filtrar los comentarios por su email, la sentencia sería la siguiente: { email: "sean_bean@gameofthron.es" }



The screenshot shows the MongoDB Compass interface. At the top, the database is 'ServidorItAcademy' and the collection is 'Ocio'. The 'Documents' tab is selected, showing 50K documents. A query filter is applied: { email: "sean_bean@gameofthron.es" }. The results are displayed in a list view, showing four documents. Each document has fields: _id, name, email, movie_id, text, and date. The email field in all documents is "sean_bean@gameofthron.es". The date field shows timestamps. The interface includes buttons for 'Generate query', 'Explain', 'Reset', 'Find', and 'Options'. A red box highlights the '1 - 25 of 273' pagination indicator.

En la consola de comando la sentencia sería de la siguiente manera, con db.comments.countDocuments para contar los documentos que contienen el email seleccionado: db.comments.countDocuments({email: "sean_bean@gameofthron.es"})

```
>_MONGOSH
> use Ocio
< switched to db Ocio
> db.comments.countDocuments({email: "sean_bean@gameofthron.es"})
< 273
Ocio>
```

Ejercicio 2

¿Cuántos cines existen en cada código postal situados dentro del estado Washington DC (DC)?

Para saber cuantos cines existen en cada código postal situado en el estado de Washington será necesario darle a la pestaña Aggregations y darle a add stage, en el primero ponemos como filtro `$match` para que coincidan los registros con state DC, la sentencia es la siguiente: `{"location.address.state": "DC"}`, luego será necesatio agregar otro stage en add stage, agrupar con `$group` y agrupar por los id de los registros seleccionados con la sentencia anterior, la sentencia para agrupar será la siguiente: `{_id: "$location.address.zipcode", totalCines: { $sum: 1 }}`

The screenshot shows the MongoDB Aggregations interface. The first stage is `$match` with the query `{"location.address.state": "DC"}`. The stage output shows three documents with their `_id`, `theaterId`, and `location` fields. The second stage is `$group` with the query `{_id: "$location.address.zipcode", totalCines: { $sum: 1 }}`. The stage output shows three documents with their `_id` (zipcodes) and `totalCines` (count).

Stage	Query	Output
\$match	<code>{"location.address.state": "DC"}</code>	Three documents with <code>_id</code> , <code>theaterId</code> , and <code>location</code> fields.
\$group	<code>{_id: "\$location.address.zipcode", totalCines: { \$sum: 1 }}</code>	Three documents with <code>_id</code> (zipcodes) and <code>totalCines</code> (count).

Para hacerlo en la consola de comandos será igual pero poniendo `db.theaters.aggregate` para hacer un filtro de agregación y poder seleccionar los registros con state DC y agruparlos por el id, la sentencia seria la siguiente: `db.theaters.aggregate([{$match: {"location.address.state": "DC"}}, {$group: {_id: "$location.address.zipcode", totalCines: {$sum: 1}}})`

The screenshot shows the MongoDB command line interface. The command `db.theaters.aggregate([{$match: {"location.address.state": "DC"}}, {$group: {_id: "$location.address.zipcode", totalCines: {$sum: 1}}})` is entered and executed. The output shows three documents with their `_id` (zipcodes) and `totalCines` (count).

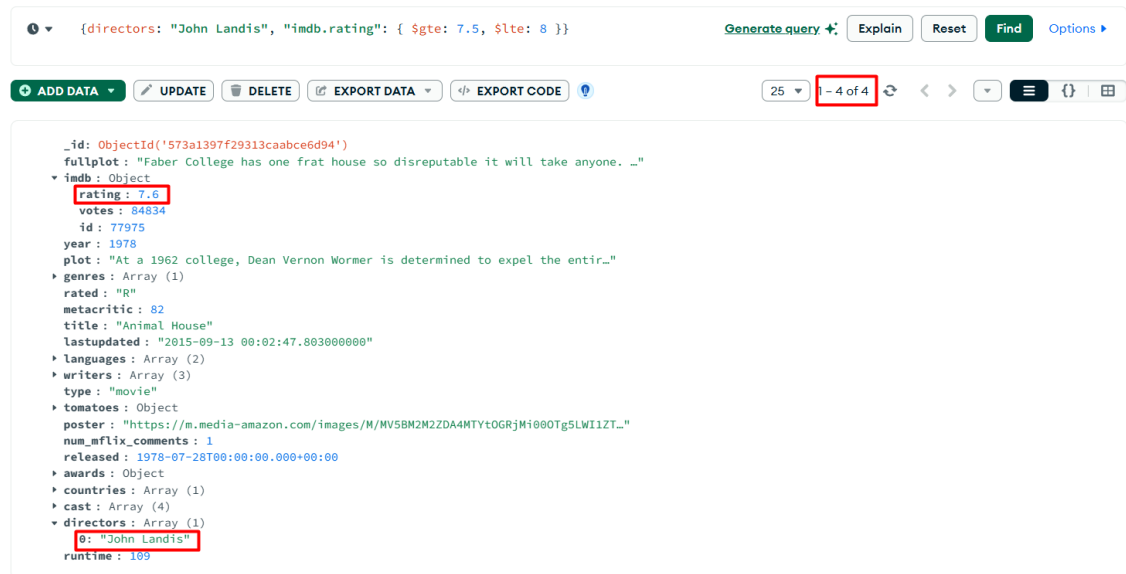
```
> use Ocio
< switched to db Ocio
> db.theaters.aggregate([{$match: {"location.address.state": "DC"}}, {$group: {_id: "$location.address.zipcode", totalCines: {$sum: 1}}})
< {
  "_id": "20016",
  "totalCines": 1
}
{
  "_id": "20010",
  "totalCines": 1
}
{
  "_id": "20002",
  "totalCines": 1
}
Ocio>
```

Nivel 3

Ejercicio 1

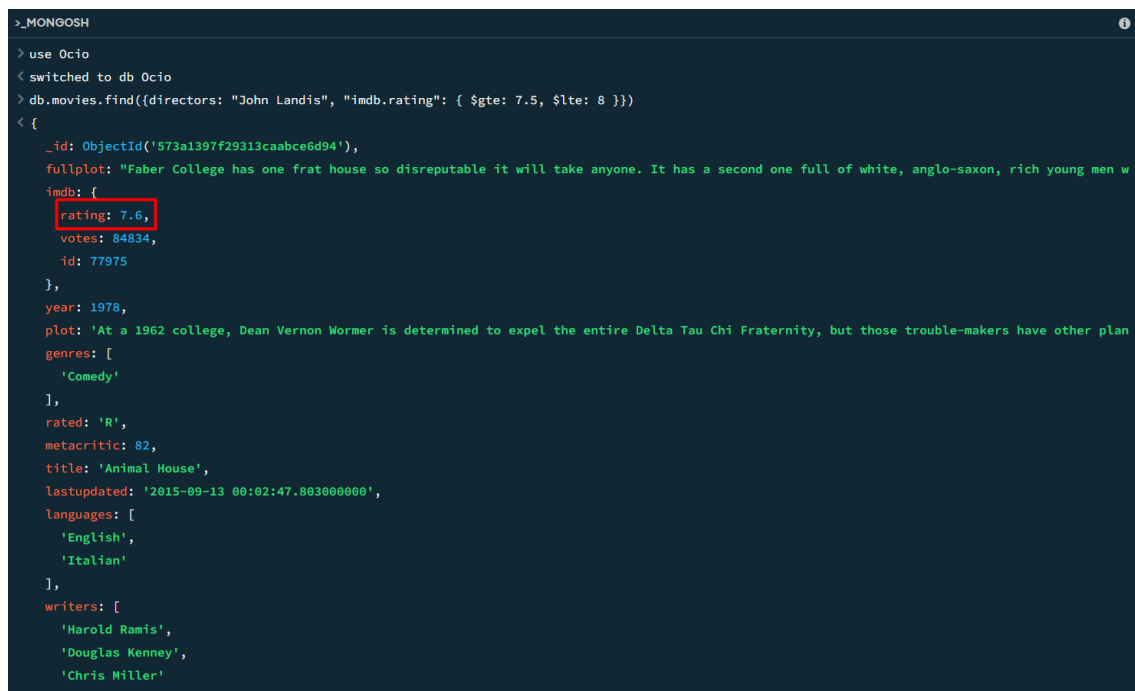
Encuentra todas las películas dirigidas por John Landis con una puntuación IMDb (Internet Movie Database) de entre 7,5 y 8.

Para encontrar todas las películas dirigidas por John Landis con las demás condiciones tendremos que filtrar directores con John Landis, y hacer un between en imdb.rating entre 7.5 y 8, la sentencia seria la siguiente: {directors: "John Landis", "imdb.rating": { \$gte: 7.5, \$lte: 8 }}



The screenshot shows the MongoDB Compass web interface. At the top, a query is entered: `{directors: "John Landis", "imdb.rating": { $gte: 7.5, $lte: 8 }}`. Below the query bar, there are buttons for "Generate query", "Explain", "Reset", "Find", and "Options". The "Find" button is highlighted. Below the buttons, there are tabs for "ADD DATA", "UPDATE", "DELETE", "EXPORT DATA", and "EXPORT CODE". The "EXPORT DATA" tab is selected. The results are displayed in a list view, showing 25 items. The first item is highlighted, and its details are shown in a sidebar. The details include: `_id: ObjectId('573a1397f29313caabce6d94')`, `fullplot: "Faber College has one frat house so disreputable it will take anyone. ..."`, `imdb: Object`, `rating: 7.6`, `votes: 84834`, `id: 77975`, `year: 1978`, `plot: "At a 1962 college, Dean Vernon Wormer is determined to expel the entire..."`, `genres: Array (1)`, `rated: "R"`, `metacritic: 82`, `title: "Animal House"`, `lastupdated: "2015-09-13 00:02:47.803000000"`, `languages: Array (2)`, `writers: Array (3)`, `type: "movie"`, `tomatoes: Object`, `poster: "https://m.media-amazon.com/images/M/MV5BM2M2ZDA4MTYtOGRjMi00OTg5LWI1ZT..."`, `num_flix_comments: 1`, `released: 1978-07-28T00:00:00.000+00:00`, `awards: Object`, `countries: Array (1)`, `cast: Array (4)`, `directors: Array (1)`, `0: "John Landis"`, and `runtime: 109`. The "rating: 7.6" and "0: "John Landis"" fields are highlighted with red boxes.

En la consola de comandos seria igual pero poniendo `db.movies.find` para buscar en movies, la sentencia seria la siguiente: `db.movies.find({directors: "John Landis", "imdb.rating": { $gte: 7.5, $lte: 8 }})`



```
>_MONGOSH
> use Ocio
< switched to db Ocio
> db.movies.find({directors: "John Landis", "imdb.rating": { $gte: 7.5, $lte: 8 }})
< {
  _id: ObjectId('573a1397f29313caabce6d94'),
  fullplot: "Faber College has one frat house so disreputable it will take anyone. It has a second one full of white, anglo-saxon, rich young men w
  imdb: {
    rating: 7.6,
    votes: 84834,
    id: 77975
  },
  year: 1978,
  plot: 'At a 1962 college, Dean Vernon Wormer is determined to expel the entire Delta Tau Chi Fraternity, but those trouble-makers have other plan
  genres: [
    'Comedy'
  ],
  rated: 'R',
  metacritic: 82,
  title: 'Animal House',
  lastupdated: '2015-09-13 00:02:47.803000000',
  languages: [
    'English',
    'Italian'
  ],
  writers: [
    'Harold Ramis',
    'Douglas Kenney',
    'Chris Miller'
  ]
}
```

Ejercicio 2

Muestra en un mapa la ubicación de todos los teatros de la base de datos.

Para mostrar un mapa en la que aparezca la ubicación de todos los teatros tendremos que darle a la pestaña Schema, y desplegar la opción location, para que aparezca el mapa con todas las ubicaciones.

theaters +

ServidoritAcademy > Ocio > theaters [Open MongoDB shell](#)

Documents 1.6K Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) [Reset](#) [Analyze](#) [Options](#)

[EXPORT SCHEMA](#) This report is based on a sample of 1000 documents. [Learn more](#)

location Document with 2 nested fields.

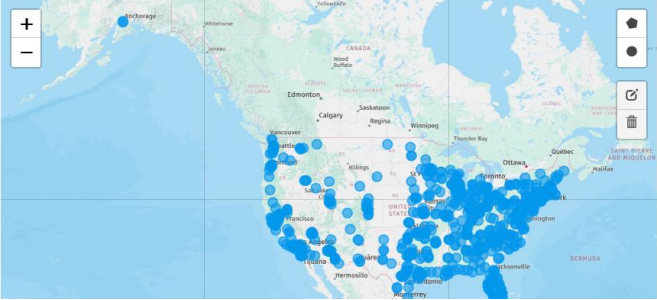
document

address Document with 5 nested fields.

document

geo

coordinates

A map of North America, including parts of Canada, the United States, and northern Mexico. Numerous blue dots are scattered across the landmass, representing the locations of theaters. The dots are more densely clustered in the eastern United States and the Great Lakes region, with fewer dots in the western United States and Canada. The map includes labels for major cities and geographical features.